Smart Budgeting and Financial Goal Management System

API Specification & Backend Contract

Official project name	Smart Budgeting and Financial Goal Management System
Prepared for	Team J
Version / date	Version 1.0 | 8 October 2026

Working draft. Confirm project-specific decisions with the team and supervisor before treating this document as approved.

1. API conventions

- Base path proposed: /api/v1/.
- Use JSON request/response bodies and conventional HTTP status codes.
- All private endpoints require authentication.
- Every object query must be scoped to the authenticated user; never trust user_id supplied by a client to set ownership.
- Use decimal strings or a documented JSON number policy for money; persist money as DECIMAL/NUMERIC, not floating point.
- Exact authentication mechanism (session or token), pagination style and error format must be selected during implementation.

2. Proposed endpoint catalogue

Method and path	Purpose	Access / notes
POST /auth/register	Create account	Public; validate unique identifier and password policy.
POST /auth/login	Authenticate	Public; do not reveal whether identifier or password was wrong.
POST /auth/logout	End session	Authenticated.
GET /me	Get current user's profile	Authenticated.
GET /categories	List categories	Authenticated; return system defaults plus user's own if adopted.
POST /categories	Create category	Authenticated; validate name and kind.
PATCH /categories/{id}	Update category	Owner-scoped; handle categories already in use.
DELETE /categories/{id}	Delete category	Owner-scoped; define behavior if referenced.
GET /transactions	List transactions	Authenticated; filters: type, category, start_date, end_date.
POST /transactions	Create transaction	Authenticated; server assigns owner.
GET /transactions/{id}	Retrieve transaction	Owner-scoped.
PATCH /transactions/{id}	Update transaction	Owner-scoped; revalidate all fields.
DELETE /transactions/{id}	Delete transaction	Owner-scoped; recalculate derived totals.
GET /budgets	List budgets	Authenticated; optionally filter by period.
POST /budgets	Create budget	Authenticated; validate category and period overlap.
PATCH /budgets/{id}	Update budget	Owner-scoped.
DELETE /budgets/{id}	Delete budget	Owner-scoped.
GET /goals	List savings goals	Authenticated.
POST /goals	Create goal	Authenticated.
PATCH /goals/{id}	Update goal	Owner-scoped.
POST /goals/{id}/contributions	Record contribution	Authenticated; goal must belong to user.

GET /goals/{id}/contributions	List contributions	Owner-scoped.
GET /dashboard?start_date=&end_date=	Dashboard totals	Authenticated; calculated from owned records.
GET /reports/summary?start_date=&end_date=	Summary report	Authenticated; validate date range.
GET /insights	Return applicable insights	Authenticated; explain rule and relevant period.

3. Example transaction payloads

Create expense request

```
{
  "type": "expense",
  "amount": "125.50",
  "transaction_date": "2026-10-08",
  "category_id": "category-id",
  "note": "Groceries"
}
```

The payload is illustrative. The server should derive the owner from the authenticated session/token rather than accept an owner ID from the request.

4. Suggested error response

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Please correct the highlighted fields.",
    "fields": {"amount": ["Amount must be greater than zero."]}
  }
}
```

5. Calculation contract

Output	Definition
income_total	Sum of income transaction amounts within inclusive selected date range.
expense_total	Sum of expense transaction amounts within inclusive selected date range.
net_cash_flow	income_total − expense_total for selected period; not a verified bank balance.
budget_used	Sum of expenses matching the budget's category and date period.
budget_remaining	limit_amount − budget_used; negative values indicate overspending.
goal_contributed	Sum of contributions linked to the goal.
goal_progress	goal_contributed ÷ target_amount × 100; cap only the displayed progress bar at 100%.

6. Decisions required before API implementation

- Choose session authentication or token authentication and define CSRF/CORS behavior as appropriate.
- Choose email, username or both as the login identifier.
- Choose UUID or integer primary keys and pagination conventions.
- Confirm default categories, custom category rules and deletion behavior.
- Confirm whether goal contributions are separate records or also linked to financial transactions; avoid double counting.
- Agree date/timezone behavior and inclusive date boundaries.